Question 9: Prepare a containerization and orchestration report describing the architecture, deployment strategy, scaling results, and observations with screenshots.

This report describes the complete containerization and orchestration workflow of Lab 2: architecture, deployment strategy, scaling results and observations.

1. Objective

Take the Lab 1 Flask application, containerize it with Docker, orchestrate its deployment on a Minikube Kubernetes cluster, and automate the whole path from a git push to a running new version through the existing Jenkins CI/CD server.

2. Tools and environment

- Ubuntu 24.04 LTS (WSL2), Docker Engine 29.9.2, Docker Compose v5.7.0
- Minikube v1.39.0 (docker driver, --force-systemd), Kubernetes v1.37.0, kubectl v1.37.2
- Jenkins 2.580.2 LTS (reused from Lab 1) with docker + kubectl access
- Docker Hub registry (abhinaaav), Python 3.12 / Flask 3.1

3. Architecture

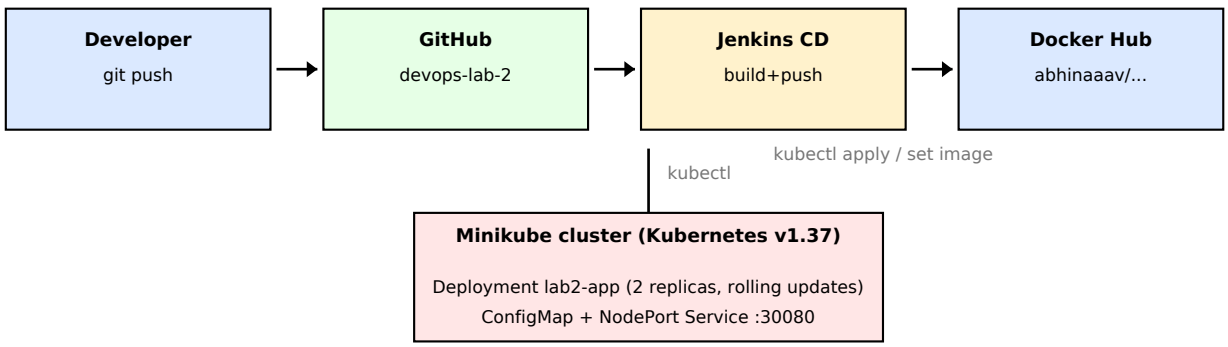


Fig: Containerization and orchestration architecture of Lab 2

The developer pushes commits to GitHub. Jenkins polls the repository (H/5), builds the image with a per-build version tag, pushes it to Docker Hub, applies the manifests with kubectl and updates the Deployment image. The Minikube cluster runs 2 replicas behind a NodePort service with readiness/liveness probes on /health.

4. Deployment strategy

- Rolling update (default strategy): kubectl set image replaces pods one at a time; maxUnavailable/maxSurge defaults keep the service available throughout.
- Rollback: kubectl rollout undo restores the previous ReplicaSet; verified by the served version reverting from 2.0.1 to 2.0.0.
- Health gating: readiness/liveness probes on /health and a final curl in the pipeline - a bad deployment fails the build instead of serving traffic.

- Versioning: images are tagged 2.1. by the CD pipeline; the version is baked into the image at build time so rollbacks visibly revert the application version.

5. Automation results

Run	Trigger	Image	Result	Cluster state after
docker run (Q1)	manual	devops-lab2-app:2.0.0	OK	container on :5000
compose up (Q2)	manual	app:2.0.0 + redis:7	OK	web + redis up
kubectl apply (Q5)	manual	devops-lab2-app:2.0.0	OK	2/2 pods, :30080
Jenkins build #1	manual	abhinaaav/...:2.1.1	FAILED (health)	rollout succeeded
Jenkins build #2	manual	abhinaaav/...:2.1.2	SUCCESS	2.1.2 live
Jenkins build #3	Poll SCM (auto)	abhinaaav/...:2.1.3	SUCCESS	2.1.3 live

Build 1 note: build, push and rollout had succeeded - only the final health check failed (see section 7 and the observations).

6. Scaling results

Command	Replicas	Observed pods
initial deployment	2	2 Running
kubectl scale --replicas=4	4	4 Running within seconds
kubectl scale --replicas=2	2	back to 2 Running, extras terminated

7. Evidence

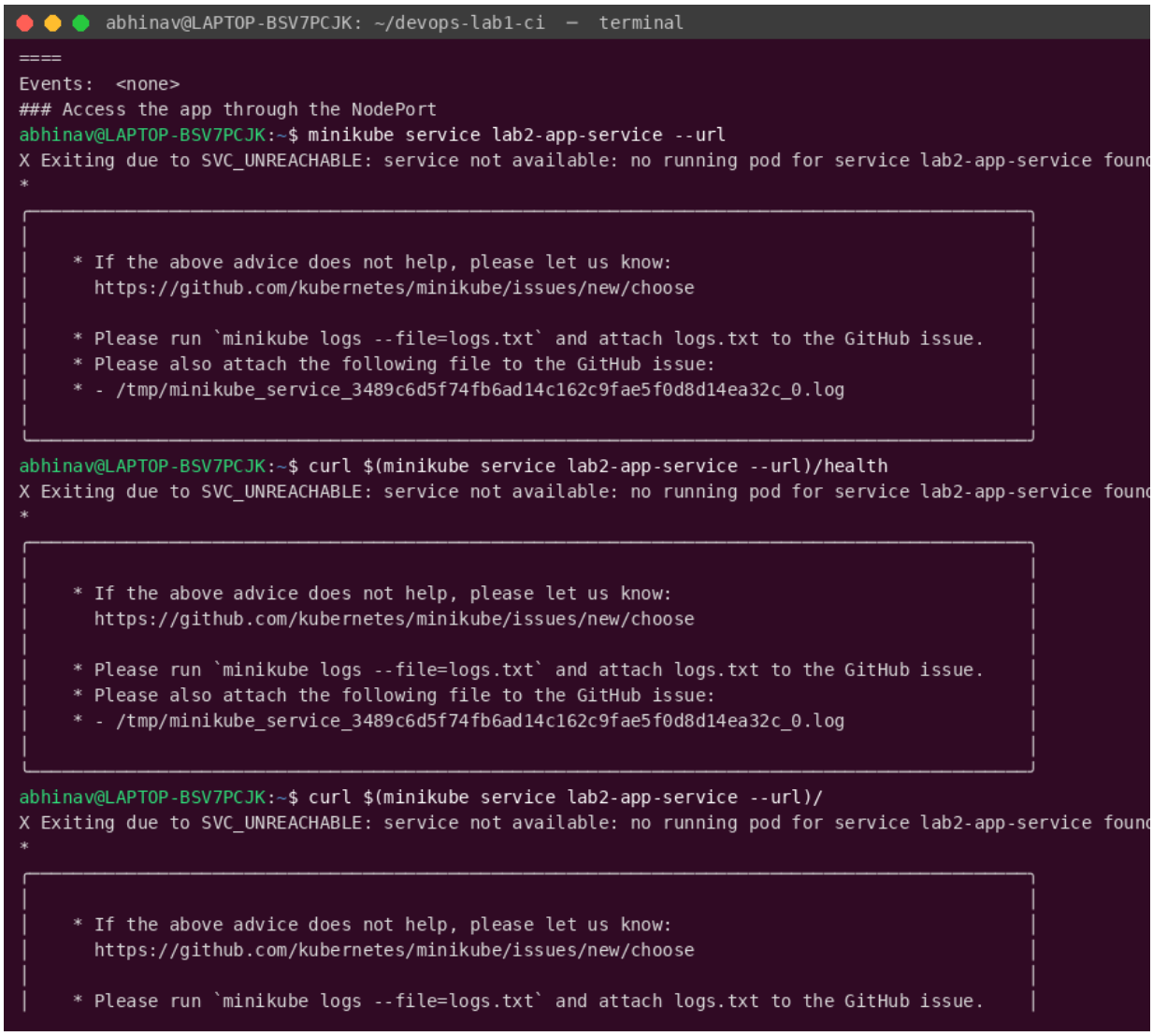


Fig: Deployment applied, rollout finished, pods running

```
abhinav@LAPTOP-BSV7PCJK: ~/devops-lab1-ci  —  terminal
lab2-app-56dd487c9b-682tp   1/1     Running       0          25s
lab2-app-88f4c9fc9-9fjc5    1/1     Terminating   0          77s
lab2-app-88f4c9fc9-kdrsn    1/1     Terminating   0          65s
abhinav@LAPTOP-BSV7PCJK:~$ curl health after rolling update:
{"status":"UP","version":"2.0.1"}
abhinav@LAPTOP-BSV7PCJK:~$ kubectl rollout history deployment/lab2-app
deployment.apps/lab2-app
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
4         <none>
### ROLLBACK: undo to the previous revision (2.0.0 image)
abhinav@LAPTOP-BSV7PCJK:~$ kubectl rollout undo deployment/lab2-app
Warning: resource deployments/lab2-app was previously managed with 'kubectl apply'. Rolling back will not
deployment.apps/lab2-app rolled back
Waiting for deployment "lab2-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "lab2-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "lab2-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "lab2-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "lab2-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab2-app" rollout to finish: 1 old replicas are pending termination...
deployment "lab2-app" successfully rolled out
abhinav@LAPTOP-BSV7PCJK:~$ kubectl get pods -l app=lab2-app
NAME                        READY   STATUS        RESTARTS   AGE
lab2-app-56dd487c9b-2ctj2   1/1     Terminating   0          59s
lab2-app-56dd487c9b-682tp   1/1     Terminating   0          72s
lab2-app-88f4c9fc9-bv4qd    1/1     Running       0          12s
lab2-app-88f4c9fc9-snrr5    1/1     Running       0          24s
abhinav@LAPTOP-BSV7PCJK:~$ curl health after rollback:
{"status":"UP","version":"2.0.1"}
--> the app is serving 2.0.0 again after the rollback
abhinav@LAPTOP-BSV7PCJK:~$ kubectl rollout history deployment/lab2-app
deployment.apps/lab2-app
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
4         <none>
```

Fig: Rolling update transitions

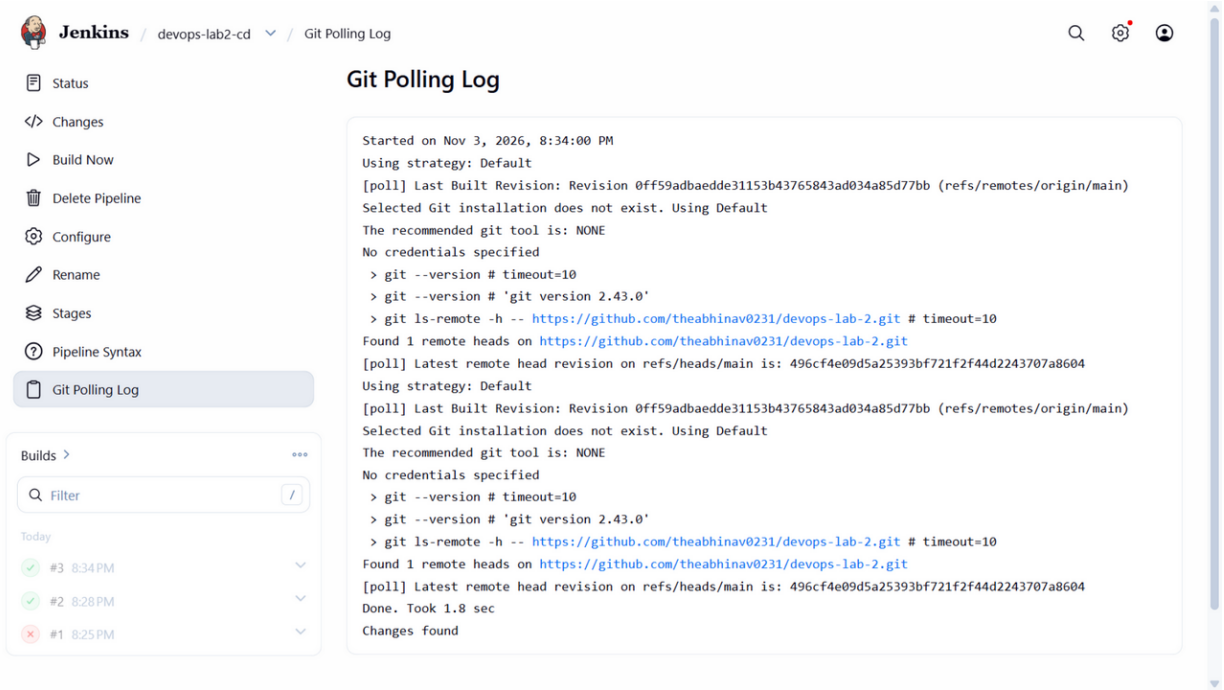


Fig: Automatic trigger: poll found changes

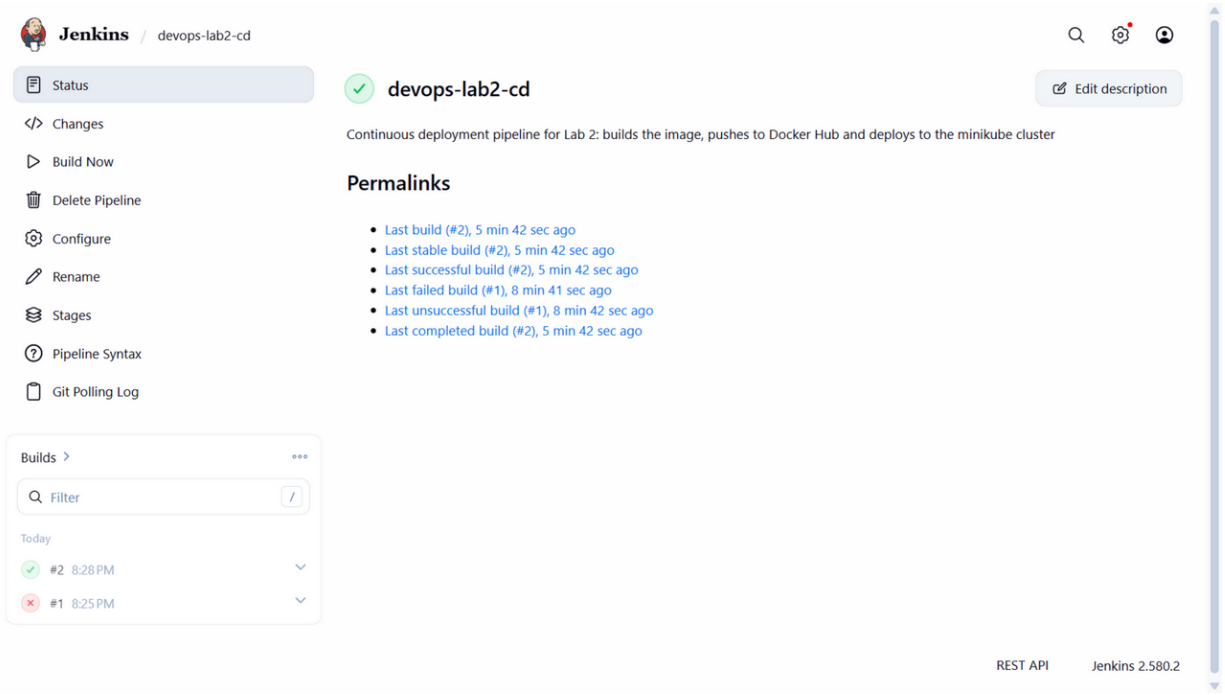


Fig: CD job: failed build 1, successful builds 2-3



Fig: Application served by the cluster

8. Observations

- Minikube on WSL2 needed --force-systemd=true (upstream issue #20469); the node container exited 255 without it.
- Docker 29.x defaults to the systemd cgroup driver which conflicts with the minikube docker driver; switching Docker to native.cgroupdriver=cgroupfs resolved the runc failure.
- With the containerd runtime, minikube image load replaces the experimental docker-env build.
- The first CD run failed only at its health check - the fix (kubectl node IP instead of minikube ip) was itself committed, showing the pipeline improving through the same flow it automates.

- Version-in-image made the rollback demo unambiguous; endpoint propagation explains the brief stale version seen right after rollout undo.

9. Conclusion

The application is fully containerized, stored in a public registry, orchestrated on Minikube with manifests under version control, and deployed continuously by Jenkins: a git push resulted in a new tagged image on Docker Hub and the cluster serving the new version - verified by the health endpoint - without any manual step.